CSE1325 – Object Oriented Programming
Marika Apostolova

THE UNIVERSITY OF TEXAS AT ARLINGTON

STUDENT'S GUIDE TO AWESOME JAVA PROJECTS

Here are a few guidelines for you so that you can write a manageable project in collaboration with your teammates.

Your Code, Not your rules

Follow a convention.

A convention is a set of rules that defines that defines how you should name your identifiers (variables, functions, etc.), how to document your code and how to structure your code.

This may seem pretty odd at first. You may find it easier to go out of the convention at first, but always remember

"Any fool can write code that a computer can understand. Good programmers write code that humans can understand."

Following a convention will make your code readable to your teammates and other people but also for me as a professor and most importantly it will lead you to maximum points for this assessment activity 😊

Assign meaningful names

As some programmers say, "Explicit is always better than implicit".

```
//You will not understand what x or myVariable
//is after a few lines of codes
int x,a,b,c,d, myVariable;//By giving a proper name you save your brain from the stress
//of remembering the meanings of all those a,b,c,d,x,y,z
int numOfTickets, userId;//Some programmers even go to the extent
```

```
//of not using any abbreviations
int numberOfTickets;
```

However it is pretty intuitive to keep the loop counters as i,j,k.

Use camel case identifiers

You know camels, right? You also know their shape?

In camel case convention you start your identifier with a lower-case letter and the first letter of each other words will be in capital.

Although there are lots of conventions to follow while naming variables, you will see how the camel case convention is the most widely followed convention by developers of almost every language.

```
//Variables should be in camelCase
int numOfTickets, userId;//Constants should be all uppercase and the
words
//will be separated by underscores
const int NUMBER_OF_SEATS = 100;
const float PI=3.14f;
```

Indentation

We have smart IDEs these days which does this step for us. But if you use an IDE from the last century, then remember, the TAB button is your best friend.

```
if( ticketBought ){
System.out.println( "Welcome passenger" );
if( isPremium ) System.out.println( "Your lunch is on us!" );
printf( "Have a nice journey!\n" );
}
```

If you write your code like this, don't blame me if you get hatred from your team mates. Always follow proper indentations in the following manner

```
if( ticketBought ){
      System.out.println ( "Welcome passenger." );
      if( isPremium ) {
```

```
                System.out.println ( "Your lunch is on us!" );
        }       System.out.println "Have a nice journey!" );
}
```

Now one can easily understand which line belongs to which block.

Also you can omit the curly braces if you have one line after an if condition, but seriously never do it. You might later unconsciously add another line there and waste hours finding the bug in your code.

/*Code Tells You How, Comments Tell You Why*/

Always put comments in your code. Human brain is very fragile, and legend says it will forget why you put a line in your code after two minutes.

Look at the following code:

```
for(i = 0; i< MAX_PASS_LENGTH - 1; i++){
        if(i<0){
                i=0;
        }
        if (ch == 13)
                break;
        if (ch == 8){
                i--;
                continue;
        }
        passA[i] = ch;
        System.out.print('*');
}
```

You see, JAVA can turn into gibberish easily. And if you write your code like this, without commenting even **PROFESSOR DUMBLEDORE CAN'T HELP YOU**!

A good example:

```
/**
* Program to find average of three numbers!
* The FindAvg program implements an application that
* simply calculates average of three integers and Prints
* the output on the screen.
*
* @author Name Surnames
*
*/
```

```java
public class Main {

    /**
    * This method is used to find average of three numbers.
    * numX This is the first parameter
    * numY  This is the second parameter
    * numZ  This is the second parameter to
    * return int This returns average of numA, numB and numC.
    */
    public int findAvg(int numX, int numY, int numZ)
    {
        return (numX + numY + numZ)/3;
    }

    /**
    * This is the main method which makes use of findAvg method.
    */

    public static void main(String args[])
    {
        Main obj = new Main();
        int avg = obj.findAvg(30, 60, 90);

       System.out.println("Average of 30, 60 and 90 is : " + avg);
    }
}
```

Commenting is good, but **don't overdo it.** The following comment is unnecessary and your peers may even find it humiliating.

```java
//Assigning 10 to the variable spam
int spam = 10;
```

Use Pure Functions

Make your code modular. Modularity increases the readability and reduces copy pasting of your code.

To maintain modularity write functions that do only one specific task. Also, the function shouldn't have access to variables.

```
public static int passengers = 10;
public static int ticketCost = 100;

	//...

	static void calculateFareCollected(){
		int total = passengers*ticketCost;
		System.out.println("Total Money Collected: " + total);
	}

	public static void main(String[] args) {
		calculateFareCollected();
		//...
	}
```

The above code is bad design. First of all the calculateFareCollected() function is not only calculating the fare but also printing it. So, you can only print the value and not use it in other functions. Also, it is accessing two global variables which reduces readability and makes your code prone to bugs. A function might silently change a global variable in a line without you noticing it which may break the whole program.

A better approach to the above code would be:

```
//...
static int calculateFareCollected( int passengers, int ticketCost){
		return passengers*ticketCost;

	}

	public static void main(String[] args) {
		int passengers = 10;
		int ticketCost = 100;
		int totalFare =
calculateFareCollected(passengers,ticketCost);

		System.out.println("Total Fare Collected " + totalFare);

		//...
```

The force of dividing a class into multiple classes

Java language is one of the most popular languages among all programming languages. There are several advantages of using the java programming language, whether for security purposes or building large distribution projects. One of the advantages of using Java is that it tries to connect every concept in the language to the real world with the help of the concepts of classes, inheritance, polymorphism, etc.

If you put everything into one class, you have to worry about everything at once when you're writing new code. This may work for small projects, but for huge applications (we're talking hundreds of thousands of lines), this quickly becomes next to impossible.

To solve this issue, you break up pieces of functionality into their own classes and encapsulate all the logic. Then when you want to work on the class, you don't need to think about what else is going on in the code. You can just focus on that small piece of code. This is invaluable for working efficiently, however it's hard to appreciate without working on applications that are huge.

Of course, there are countless other benefits to breaking your code into smaller pieces: the code is more maintainable, more testable, more reusable, etc., but to me the biggest benefit is that it makes massive programs manageable by reducing the amount of code you need to think about at one time. Custom classes are created by the user for a particular purpose.

For example, in a Ticket Management type of application you may need a class definition of a Vehicle and another class for a User. So, you can have your user class and all the functions that process a user defined in a class called user
Similarly, you put all the vehicle related definitions in a class called vehicle
This way, you can assign each file to each of your group member.
Then finally put all the files in the same directory and merge them in a main class.

Make sure that only the main class has a main() function. However, during development, each developer can write their own main function to test their codes and remove them before merging.

```
public class Main {

	public static void main(String[] args)

	{
	TestSystem.user.print1();//a method call from the same package but another class
	TestSystem.vehicle.print2();
	TestSystem2.brand.print3();();//a method call from another package
	}
}
```

Project design

There is a famous saying among developers that goes

"A week of coding can save you from hours of planning."

Make sure you plan the workflow and project design before getting into coding. Remember coding is the easiest part of making a project. The hard stones are the designing and maintenance.

Make a diagram (Figure 1) of which page will come after which. Which key press will bring you to which page. It is good for your project documentation.

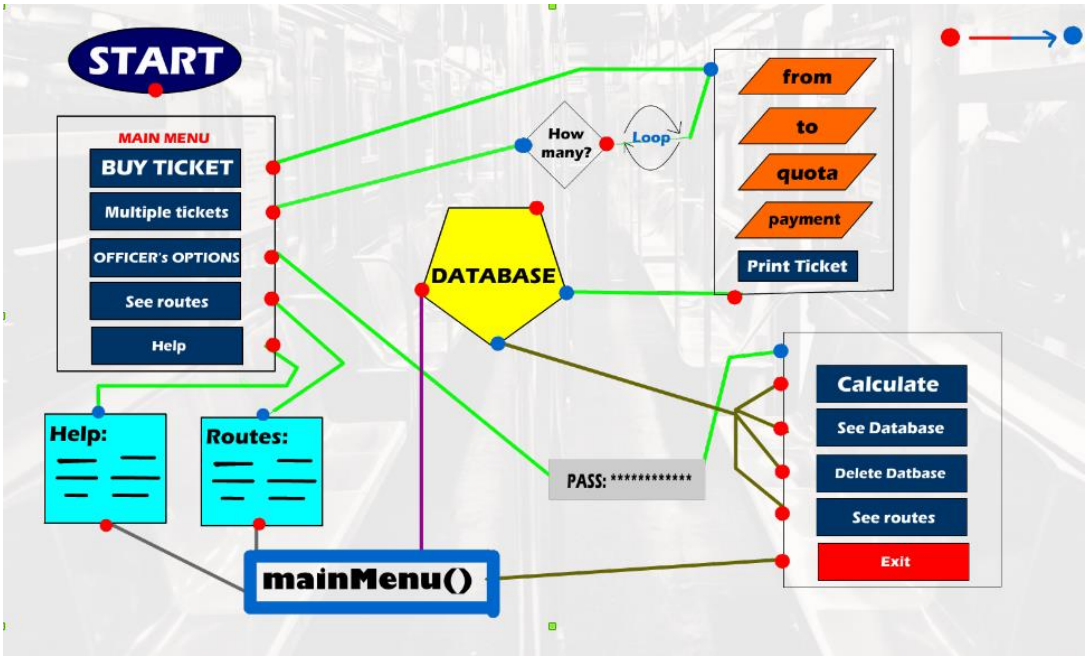


Figure 1 - A students' flowchart of a project

You are just starting out, and I assume you don't know about design patterns, database schemas yet. So, a basic flowchart like the one above would be enough to communicate with your team members.

Before starting the function definitions, define the function signatures and what each of them will do. This will help each member to work independently on his portion.

You can also use UML!

Files are your best friends

Since JAVA is mostly an introductory course, you probably have no idea about any database system. So, make sure you know about files the most. They are your best options to store information.

Study about reading and writing to files. The basic concepts stay in other programming languages as well, and you will thank yourself later.

If you only know how to read variables from a file, you should also see how to read or write entire data in a file.

```
import java.io.File;
import java.io.FileWriter;
import java.io.IOException;

public class Main {

      public static void main(String[] args) {
            try {
                  File myObj = new File("filename.txt");
                  if (myObj.createNewFile()) {
                    System.out.println("File      created:      "      +
myObj.getName());
                  } else {
                    System.out.println("File already exists.");
                  }
                } catch (IOException e) {
                  System.out.println("An error occurred.");
                  e.printStackTrace();
                }
            try {
                  FileWriter myWriter = new FileWriter("filename.txt");
                  myWriter.write("Files in Java might be tricky, but it
is fun enough!");
                  myWriter.close();
                  System.out.println("Successfully wrote to the file.");
                } catch (IOException e) {
                  System.out.println("An error occurred.");
                  e.printStackTrace();
                }

            }
```

Wrapping up

It's 2018, and internet is more accessible than ever. Always look for tutorials and solutions. There are plenty of people that would gladly help you if you ever face any problem. Follow the mentioned guidelines to make the best possible project.

GOOD LUCK!